

OS LAB WEEK 4

a) Rate monotonic scheduling (RMS) Algorithm

```
#include <stdio.h>
#include <math.h>

#define MAX 10

int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int find_lcm(int arr[], int n) {
    int res = arr[0];
    for (int i = 1; i < n; i++) {
        res = lcm(res, arr[i]);
    }
    return res;
}

int main() {
    printf("1WA24CS242\n");
    int n;
    int C[MAX], P[MAX], A[MAX];
    int remaining[MAX] = {0};

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the CPU burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &C[i]);
    }
```

```

printf("Enter the time periods:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &P[i]);
}

printf("Enter arrival times:\n");
for (int i = 0; i < n; i++) {
    scanf("%d", &A[i]);
}

int hyper = find_lcm(P, n);
printf("LCM=%d\n\n", hyper);

float U = 0.0;
for (int i = 0; i < n; i++) {
    U += (float)C[i] / (float)P[i];
}

float bound;
bound = (float)n * (pow(2.0, 1.0 / (float)n) - 1.0);

printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i + 1, C[i], P[i]);
}

printf("\n%f <= %f => %s\n", U, bound, (U <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyper);

int last = -2;

for (int t = 0; t < hyper; t++) {

    for (int i = 0; i < n; i++) {
        if (t >= A[i] && (t - A[i]) % P[i] == 0) {
            remaining[i] = C[i];
        }
    }

    int sel = -1;
    int minP = 9999;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0 && P[i] < minP) {

```

```

        minP = P[i];
        sel = i;
    }
}

if (sel != last) {
    if (sel == -1)
        printf("%dms onwards: CPU is idle\n", t);
    else
        printf("%dms onwards: Process %d running\n", t, sel + 1);
    last = sel;
}

if (sel != -1)
    remaining[sel]--;
}

return 0;
}

```

1WA24CS242

Enter the number of processes: 2

Enter the CPU burst times:

20 35

Enter the time periods:

50 100

Enter arrival times:

0 0

LCM=100

Rate Monotone Scheduling:

PID	Burst	Period
1	20	50
2	35	100

0.750000 <= 0.828427 => true

Scheduling occurs for 100 ms

0ms onwards: Process 1 running

20ms onwards: Process 2 running

50ms onwards: Process 1 running

70ms onwards: Process 2 running

75ms onwards: CPU is idle

Process returned 0 (0x0) execution time : 24.646 s

Press any key to continue.

b) Earliest Deadline first (EDF) pseudocode

```
#include <stdio.h>

#define MAX 10

int main() {
    printf("1WA24CS242\n");
    int n;
    int burst[MAX], period[MAX], deadline[MAX];
    int remaining[MAX], abs_deadline[MAX];

    int time = 0;
    int hyper = 20;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter Burst, Deadline, Period:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
        remaining[i] = 0;
        abs_deadline[i] = deadline[i];
    }

    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\n", i + 1, burst[i], deadline[i], period[i]);
    }

    printf("\nScheduling occurs for %d ms\n\n", hyper);

    while (time < hyper) {

        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
                remaining[i] = burst[i];
                abs_deadline[i] = time + deadline[i];
            }
        }

        int selected = -1;
        int earliest_deadline = 9999;

        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                if (abs_deadline[i] < earliest_deadline) {
                    earliest_deadline = abs_deadline[i];
                }
            }
        }

        if (selected != -1) {
            remaining[selected]--;
            if (remaining[selected] == 0) {
                printf("%d\n", selected + 1);
            }
        }

        time++;
    }
}
```

```
        selected = i;
    }
}

if (selected == -1) {
    printf("%dms : CPU is idle.\n", time);
} else {
    printf("%dms : Task %d is running.\n", time, selected + 1);
    remaining[selected]--;
}

time++;
}

return 0;
```

}

1WA24CS242

Enter number of processes: 3

Enter Burst, Deadline, Period:

2 4 5

3 7 20

2 8 10

PID	Burst	Deadline	Period
1	2	4	5
2	3	7	20
3	2	8	10

Scheduling occurs for 20 ms

0ms : Task 1 is running.

1ms : Task 1 is running.

2ms : Task 2 is running.

3ms : Task 2 is running.

4ms : Task 2 is running.

5ms : Task 3 is running.

6ms : Task 3 is running.

7ms : Task 1 is running.

8ms : Task 1 is running.

9ms : CPU is idle.

10ms : Task 1 is running.

11ms : Task 1 is running.

12ms : Task 3 is running.

13ms : Task 3 is running.

14ms : CPU is idle.

15ms : Task 1 is running.

16ms : Task 1 is running.

17ms : CPU is idle.

18ms : CPU is idle.

19ms : CPU is idle.

Process returned 0 (0x0) execution time : 47.850 s

Press any key to continue.